

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 – Redes

Sección 20

Jorge Yass



Proyecto 2 – Fase 1

Ana Laura Tschen 221645

Fernando Rueda 23748

Luis Felipe Aguilar Portillo 23195

Fernando Hernández 23645

Esteban Cárcamo 23016

Guatemala, 6 de octubre 2026

Organización del equipo

El trabajo se dividió por áreas para que cada integrante se encargara de una parte del diseño y la implementación de la red.

Integrante	Carné	Rol	Aportes
Ana Laura Tschen	221645	Diseño y documentación	Diagrama de red y redacción de la propuesta técnica.
Fernando Hernández	23645	Infraestructura de red	Configuración inicial de Terraform, VPC, subredes CIDR/VLSM, Internet Gateway y tablas de rutas.
Luis Felipe Aguilar Portillo	23195	Seguridad y cómputo	Security Groups por segmento, instancias EC2 y outputs de Terraform.
Fernando Rueda	23748	Despliegue y pruebas	Documentación del despliegue, implementación en AWS, pruebas de conectividad y capturas.
Esteban Cárcamo	23016	Segmentación	Cálculos de subneteo CIDR/VLSM (Excel) y README del repositorio.

Tabla 1. Roles y aportes de cada integrante

Nombre de la empresa

El nombre Barrilete Ventures representa la misión de impulsar a startups guatemaltecas para que alcancen nuevas oportunidades de crecimiento. El barrilete simboliza creatividad, visión y progreso, además de mantener una conexión con la identidad cultural de Guatemala. La palabra Ventures comunica su relación con el emprendimiento, la innovación y la inversión en empresas emergentes. En conjunto, el nombre refleja una organización que brinda capital, orientación y apoyo estratégico para ayudar a nuevos proyectos a llegar más lejos.

Giro del negocio

Para efectos de este proyecto, Barrilete Ventures se modela como una empresa guatemalteca dedicada a la inversión ángel y al capital de riesgo. Su actividad consiste en identificar startups en etapas tempranas, evaluar oportunidades de inversión y brindar capital, mentoría y acompañamiento estratégico para apoyar su crecimiento en Guatemala y América Latina.

Nombre de dominio

El dominio seleccionado es barrilete.dev. La extensión .dev refuerza la identidad digital de la empresa y su relación con el desarrollo, la innovación y el ecosistema tecnológico. Además, proporciona un nombre breve y fácil de asociar con la plataforma web.

Relación del negocio con la red propuesta

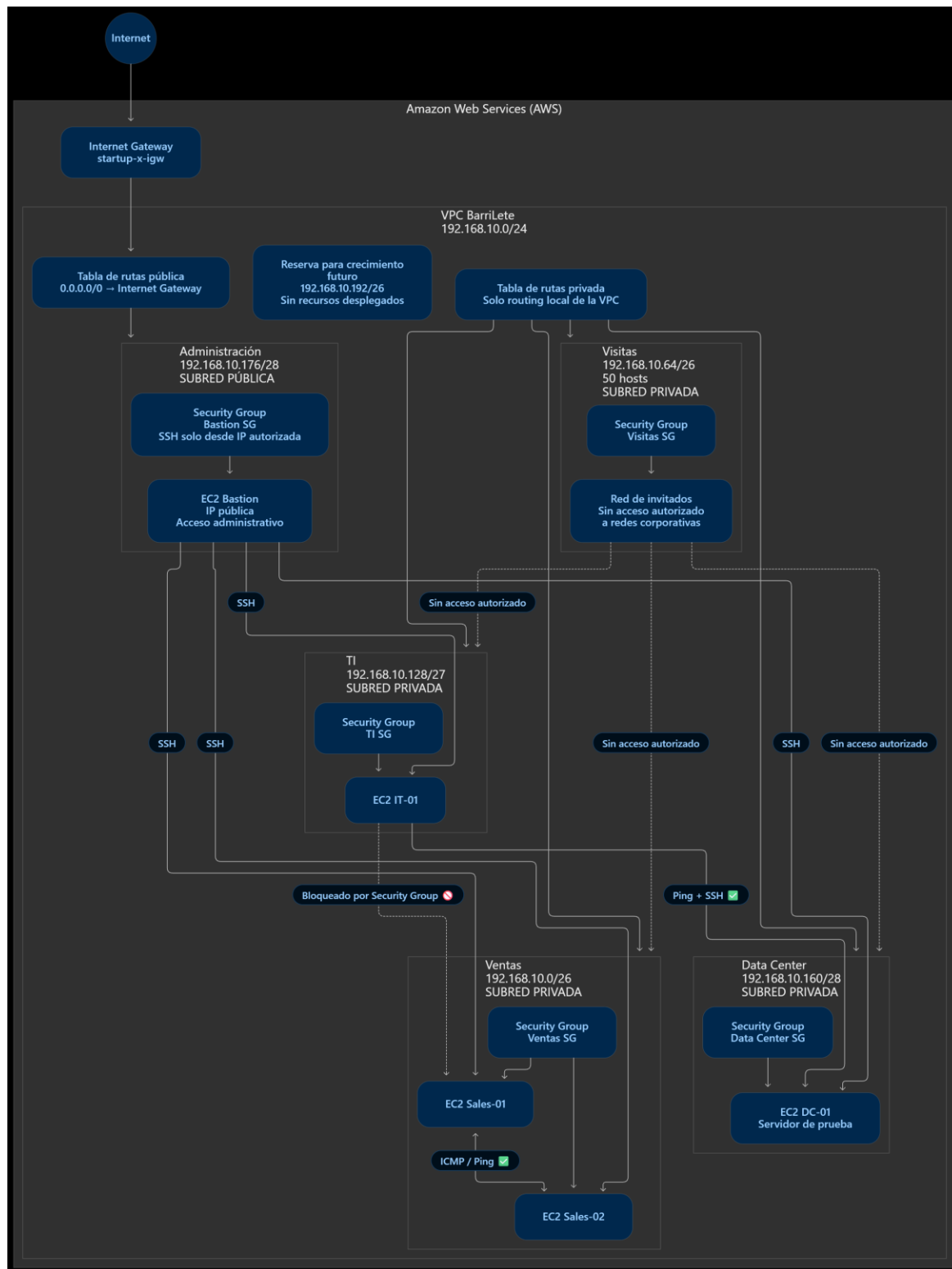
- Ventas (25 usuarios): contacto con emprendedores e inversionistas, seguimiento de oportunidades y gestión comercial.
- TI (15 usuarios): soporte de la plataforma, administración de usuarios, seguridad y operación de los servicios en la nube.
- Data Center (5 servidores): capacidad prevista para el sitio web, aplicación, base de datos, documentos y monitoreo.
- Visitas (50 usuarios): acceso aislado para emprendedores, inversionistas y proveedores, sin comunicación directa con las redes privadas.

Justificación del Proveedor Elegido

Utilizamos Amazon Web Services (AWS) como proveedor de nube. La red se implementó con Amazon Virtual Private Cloud (Amazon VPC), donde se creó el bloque 192.168.10.0/24 con las subredes de Ventas, Visitas, TI, Data Center y Administración, y con Amazon EC2 se crearon las instancias para las pruebas de conectividad.

Se eligió AWS por tres razones principales. Primero, el equipo contaba con créditos de uso en AWS, por lo que se decidió aprovecharlos para implementar la red sin incurrir en costos adicionales ni abrir cuentas en otro proveedor. Segundo, Amazon VPC permite dividir la red en subredes con su propio bloque CIDR, tablas de rutas y Security Groups, lo que facilita separar Ventas, TI, Data Center y Visitas, y decidir qué tráfico puede pasar entre ellas. En otros proveedores, como Digital Ocean, cada VPC es un único rango privado sin subredes internas, por lo que cada departamento tendría que ser una VPC separada; en AWS todos los segmentos viven dentro de una sola VPC y el tráfico entre ellos se controla con Security Groups. Tercero, AWS tiene soporte completo en Terraform, lo que permitió definir toda la infraestructura como código, crearla y destruirla cuando fuera necesario, y así no gastar los créditos más rápido de lo necesario. Además, Amazon Route 53 permitirá administrar el dominio barrilete.dev cuando se publique la página web en las siguientes fases.

Diagrama de red



Cálculos de segmentación de la red

Para la implementación del proyecto se utilizará segmentación CIDR/VLSM. Esta alternativa permite asignar a cada departamento una subred de tamaño acorde con sus necesidades, en lugar de asignar bloques del mismo tamaño a todas las áreas. La segmentación tradicional mediante cuatro redes /26 es funcional, pero desperdicia una cantidad considerable de direcciones, especialmente en el Data Center, que actualmente únicamente requiere cinco servidores.

La red base de la VPC es 192.168.10.0/24, con 256 direcciones (de 192.168.10.0 a 192.168.10.255). Antes de calcular el tamaño de cada subred se tomaron en cuenta dos restricciones de Amazon VPC:

- AWS reserva 5 direcciones en cada subred, no 2: la dirección de red, la .1 (router de la VPC), la .2 (servidor DNS), la .3 (reservada para uso futuro) y la de broadcast. Por eso, una subred con h bits de host tiene $2^h - 5$ IP utilizables.
- AWS solo permite subredes con prefijos entre /16 y /28. Una subred /29, como la que bastaría para el Data Center, no se puede crear.

Paso 1. Tamaño de cada subred

Se ordenaron los segmentos de mayor a menor necesidad y, para cada uno, se buscó el menor número de bits de host (h) que cumpliera $2^h - 5 \geq \text{hosts requeridos}$, respetando el mínimo /28 de AWS. El prefijo se obtiene como $/32 - h$.

Segmento	Hosts requeridos	Cálculo	Bits de host (h)	Prefijo	Máscara
Visitas	50	$2^5 - 5 = 27$ no alcanza; $2^6 - 5 = 59 \geq 50$	6	/26	255.255.255.192
Ventas	25	$2^5 - 5 = 27 \geq 25$, pero solo deja 2 IP libres; se usa $2^6 - 5 = 59$	6	/26	255.255.255.192
TI	15	$2^4 - 5 = 11$ no alcanza; $2^5 - 5 = 27 \geq 15$	5	/27	255.255.255.224
Data Center	5	Con /29 bastaría ($2^3 - 2 = 6$), pero el mínimo de AWS es /28: $2^4 - 5 = 11 \geq 5$	4	/28	255.255.255.240
Administración	1 (Bastion)	Mínimo de AWS: $2^4 - 5 = 11 \geq 1$	4	/28	255.255.255.240
Reserva	—	Espacio restante del /24: $2^6 = 64$ direcciones	6	/26	255.255.255.192

Tabla 2. Tamaño de cada subred

Paso 2. Asignación de los bloques

Las subredes se asignaron de forma contigua a partir de 192.168.10.0, de la más grande a la más pequeña, para que cada bloque inicie en un múltiplo de su tamaño. El tamaño del bloque es 256 menos el último octeto de la máscara: $256 - 192 = 64$ para /26, $256 - 224 = 32$ para /27 y $256 - 240 = 16$ para /28. Ventas y Visitas tienen el mismo tamaño (/26), por lo que el orden entre ellas no afecta la alineación; se colocó Ventas primero.

Así, Ventas ocupa de .0 a .63, Visitas de .64 a .127, TI de .128 a .159, Data Center de .160 a .175, Administración de .176 a .191 y la reserva de .192 a .255. Cada subred inicia en la dirección siguiente al broadcast de la anterior, por lo que ninguna se traslapa con otra.

Paso 3. Cálculo binario de la dirección de red y de broadcast

La dirección de red se obtiene con la operación AND entre una dirección IP de la subred y la máscara, y la de broadcast con la operación OR entre la dirección de red y el inverso de la máscara; el valor entre paréntesis es el decimal del octeto. Como todas las máscaras tienen 255 (1111111) en los tres primeros octetos, estos se mantienen en 192.168.10 y el cálculo solo cambia en el cuarto octeto. En la tabla se toma como ejemplo la IP de una instancia de cada subred. El desarrollo completo de los 32 bits está en el archivo de Excel de segmentación (hoja CIDR).

Segmento	IP de ejemplo	IP (4.º octeto)	Máscara (4.º octeto)	AND = red	Inverso de la máscara	OR = broadcast
Ventas	192.168.10.10 (Sales-01)	00001010 (10)	11000000 (192)	00000000 (0)	00111111 (63)	00111111 (63)
Visitas	192.168.10.74 (Guest-01, opcional)	01001010 (74)	11000000 (192)	01000000 (64)	00111111 (63)	01111111 (127)
TI	192.168.10.138 (IT-01)	10001010 (138)	11100000 (224)	10000000 (128)	00011111 (31)	10011111 (159)
Data Center	192.168.10.164 (DC-01)	10100100 (164)	11110000 (240)	10100000 (160)	00001111 (15)	10101111 (175)
Administración	192.168.10.180 (Bastion)	10110100 (180)	11110000 (240)	10110000 (176)	00001111 (15)	10111111 (191)
Reserva	192.168.10.192 (sin instancias)	11000000 (192)	11000000 (192)	11000000 (192)	00111111 (63)	11111111 (255)

Tabla 3. Cálculo binario del cuarto octeto

Resultado de la segmentación

Segmento	Dirección de red	Máscara	Broadcast	Rango utilizable en AWS	IP válidas (2 ^h - 2)	IP utilizables en AWS (2 ^h - 5)	Hosts requeridos
Ventas	192.168.10.0/26	255.255.255.192	192.168.10.63	192.168.10.4 – 192.168.10.62	62	59	25
Visitas	192.168.10.64/26	255.255.255.192	192.168.10.127	192.168.10.68 – 192.168.10.126	62	59	50
TI	192.168.10.128/27	255.255.255.224	192.168.10.159	192.168.10.132 – 192.168.10.158	30	27	15
Data Center	192.168.10.160/28	255.255.255.240	192.168.10.175	192.168.10.164 – 192.168.10.174	14	11	5
Administración	192.168.10.176/28	255.255.255.240	192.168.10.191	192.168.10.180 – 192.168.10.190	14	11	1 (Bastion)
Reserva	192.168.10.192/26	255.255.255.192	192.168.10.255	192.168.10.196 – 192.168.10.254	62	59	Crecimiento futuro

Tabla 4. Segmentación final de la red 192.168.10.0/24

Las cinco subredes que se crean en AWS ocupan 192 de las 256 direcciones (75 %), y el bloque 192.168.10.192/26 (64 direcciones, 25 %) queda libre para crecimiento futuro. Las IP privadas de las instancias de prueba están dentro del rango utilizable de su subred: Sales-01 (.10) y Sales-02 (.11) en Ventas, IT-01 (.138) en TI, DC-01 (.164) en Data Center y el Bastion (.180) en

Administración. Con cuatro redes /26 de la segmentación tradicional se ocuparía todo el /24 y no quedaría espacio para la subred de Administración ni para crecer.

Cambios respecto a la segmentación inicial

La primera propuesta usaba la red 192.168.1.0/24 con Visitas /26, Ventas /27, TI /27 y Data Center /29, calculada con la regla clásica de $2^h - 2$ IP válidas. Al implementarla en AWS se ajustó por tres motivos:

- a) El Data Center pasó de /29 a /28, porque AWS no permite crear subredes menores a /28.
- b) Ventas pasó de /27 a /26: en AWS un /27 deja 27 IP utilizables, apenas 2 más que las 25 personas del equipo, sin margen para crecer.
- c) Se agregó la subred de Administración (192.168.10.176/28). No es un departamento: aloja el Bastion, la única instancia con IP pública y el único punto de entrada por SSH hacia las subredes privadas.

Además, se usa como red base 192.168.10.0/24, que es el bloque configurado en la VPC.

Screenshots y explicación de la implementación

Pasos de la implementación

La infraestructura no se creó a mano desde la consola de AWS, sino con Terraform, una herramienta de infraestructura como código. Todos los recursos que aparecen en las capturas están descritos en archivos de configuración (.tf) que se entregan junto con este documento. Así, la red se puede crear y destruir siempre de la misma forma con un par de comandos, lo que nos permitió apagar todo después de cada prueba para no consumir créditos.

Los archivos se organizaron por tipo de recurso:

Archivo	Qué define
versions.tf y provider.tf	Versión de Terraform, proveedor AWS, región us-east-1 y etiquetas comunes de todos los recursos.
variables.tf, locals.tf y terraform.tfvars.example	Los bloques CIDR de cada subred, el tipo de instancia (t3.micro), el nombre de la llave SSH y la IP pública desde la que se permite SSH.
vpc.tf	La VPC barrilete-vpc con el bloque 192.168.10.0/24.
subnets.tf	Las cinco subredes en la zona us-east-1a. Solo la de Administración asigna IP pública.
internet-gateway.tf	El Internet Gateway barrilete-igw.
route-tables.tf	barrilete-public-rt (ruta 0.0.0.0/0 hacia el IGW), asociada solo a Administración, y barrilete-private-rt (solo la ruta local), asociada a Ventas, Visitas, TI y Data Center.
security-groups.tf	Un Security Group por segmento y uno para el Bastion.

ec2.tf	Las instancias Amazon Linux 2023 con IP privada fija.
outputs.tf	Muestra al final las IP de las instancias y el comando SSH para conectarse.

Tabla 5. Archivos de configuración de Terraform

Los pasos que seguimos fueron:

1. Preparación: creamos un usuario IAM con su clave de acceso y la configuramos en el equipo (~/.aws/credentials). En EC2 creamos el par de llaves barrilete-key (tipo ED25519) para entrar por SSH. Luego copiamos terraform.tfvars.example a terraform.tfvars y colocamos nuestra IP pública con máscara /32.
2. terraform init: descarga el proveedor de AWS y prepara la carpeta de trabajo.
3. terraform plan: muestra todo lo que se va a crear sin modificar nada en AWS. Revisamos que fueran 25 recursos.
4. terraform apply: crea los recursos respetando sus dependencias: primero la VPC, después las subredes y el Internet Gateway, luego las tablas de rutas con sus asociaciones, los Security Groups y por último las instancias. El proceso toma alrededor de tres minutos.
5. Verificación: revisamos en la consola de AWS que todo se creara correctamente (capturas de esta sección) y realizamos las pruebas de conectividad.
6. terraform destroy: al terminar eliminamos todos los recursos para no generar costos.

El control del tráfico entre segmentos se hace con los Security Groups. Todas las subredes pueden enrutarse entre sí por la ruta local de la VPC, pero un Security Group solo deja pasar lo que tiene una regla explícita y bloquea todo lo demás:

Security Group	Tráfico de entrada permitido
barrilete-bastion-sg	SSH (puerto 22) solo desde la IP pública del administrador.
barrilete-sales-sg	SSH desde el Bastion; ICMP (ping) solo entre instancias de Ventas.
barrilete-it-sg	SSH desde el Bastion.
barrilete-datacenter-sg	SSH desde el Bastion y desde TI; ICMP solo desde TI.
barrilete-guest-sg	Ninguno.

Tabla 6. Reglas de entrada de los Security Groups

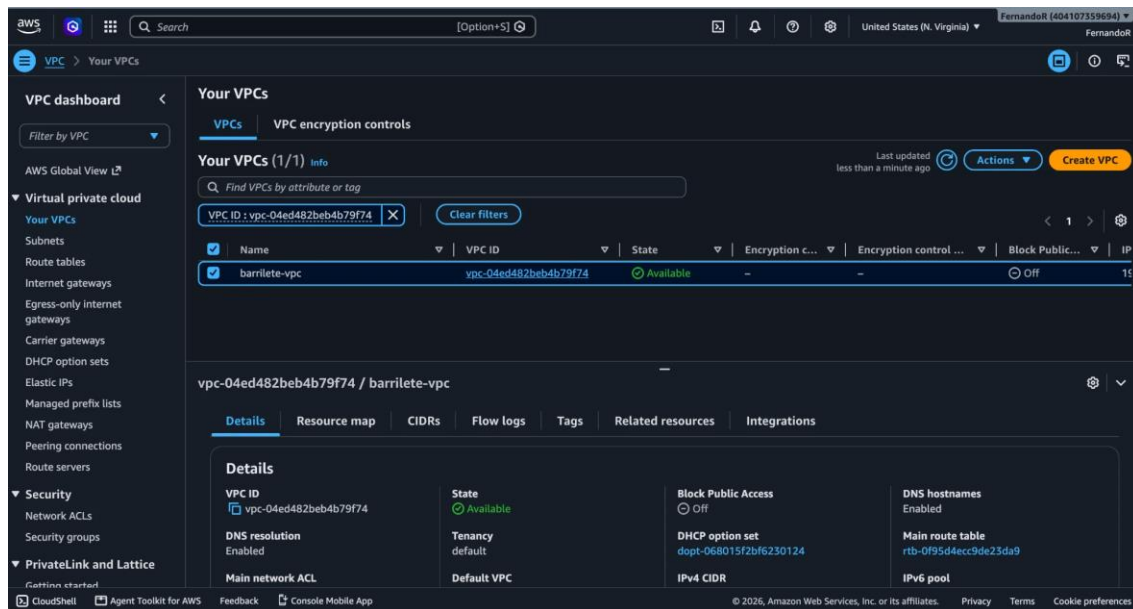
Acceso a las instancias privadas

Ventas, Visitas, TI y Data Center son subredes privadas: sus instancias no tienen IP pública ni ruta hacia Internet. Para entrar a ellas usamos el Bastion como punto de salto. Primero cargamos la llave con ssh-add barrilete-key.pem y después nos conectamos con ssh -J ec2-user@<IP pública del Bastion> ec2-user@<IP privada de destino>. Para entrar desde TI al Data Center agregamos la opción -A, que reenvía la llave sin copiarla al servidor.

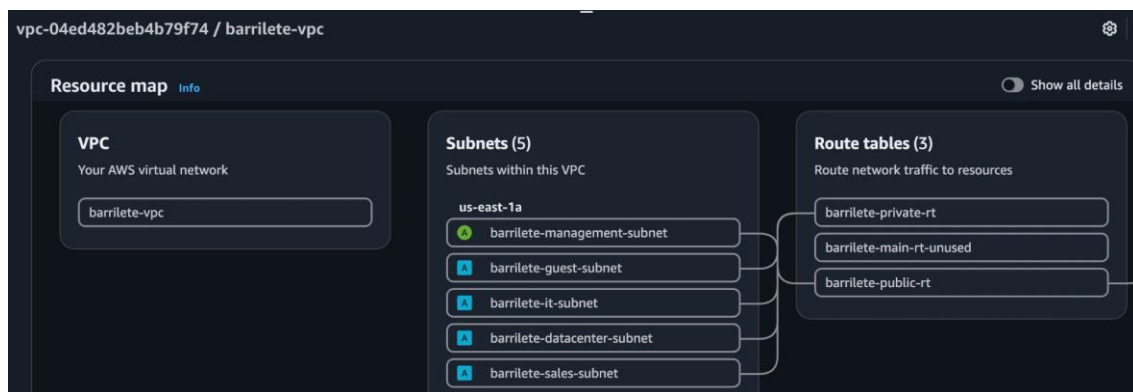
Las IP privadas de las instancias están fijas en Terraform (Sales-01 192.168.10.10, Sales-02 192.168.10.11, IT-01 192.168.10.138, DC-01 192.168.10.164 y Bastion 192.168.10.180), por lo que son las mismas cada vez que se despliega la red. La IP pública del Bastion sí cambia en cada

despliegue: en las pruebas aparece 3.235.87.197 y en la captura de instancias 3.228.3.81, porque el ambiente se volvió a crear entre ambas tomas.

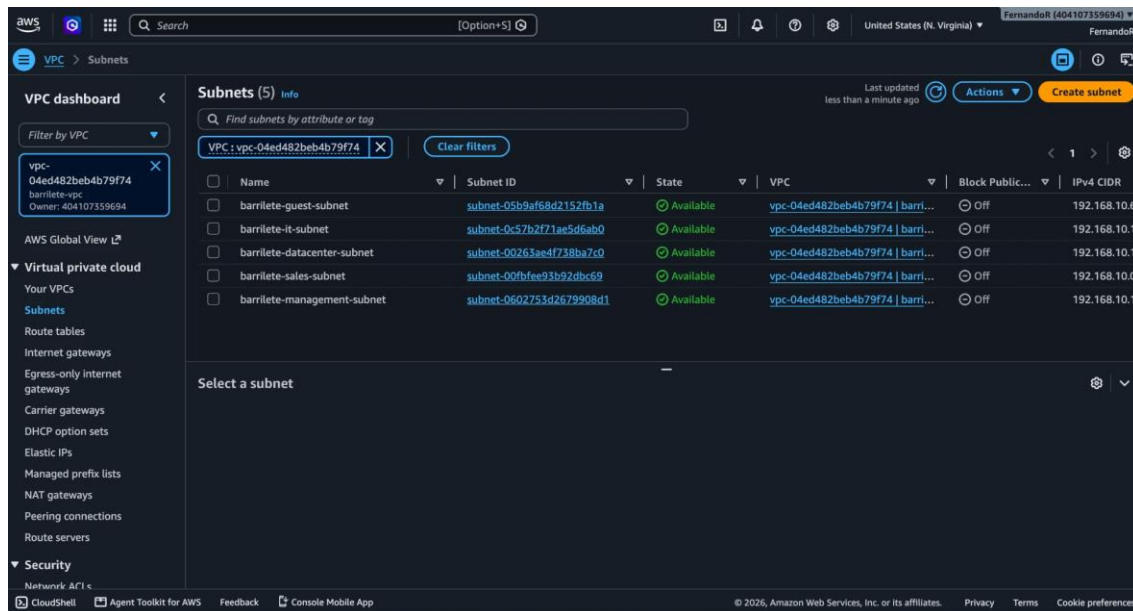
Esta es la VPC barrilete-vpc, ya creada y en estado Available. Es la red base donde vive todo el proyecto, con el bloque 192.168.10.0/24.



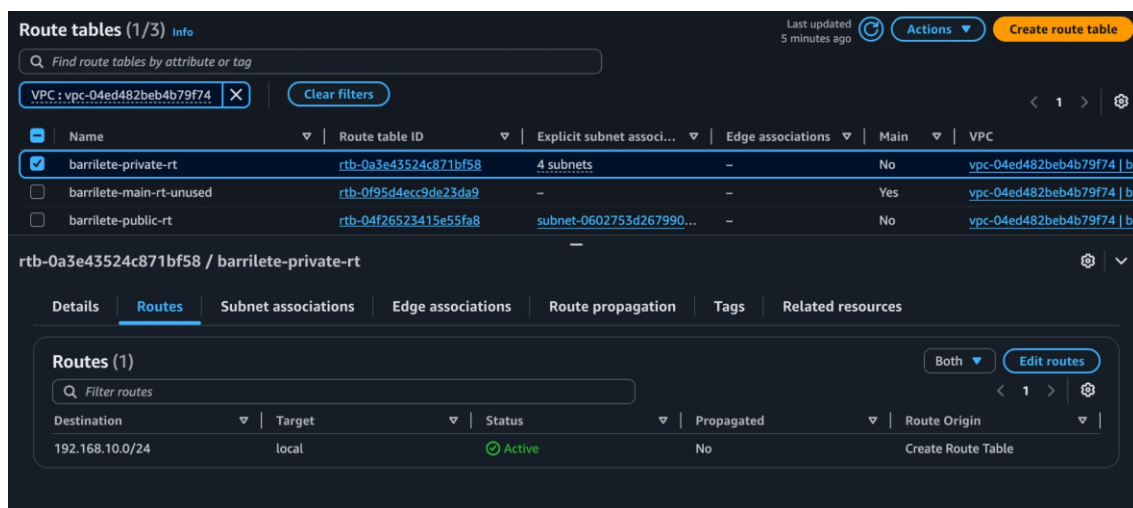
En el resource map se ve cómo queda todo conectado: las 5 subredes (management, guest, it, datacenter y sales) en us-east-1a y las 3 tablas de rutas a las que están asociadas.



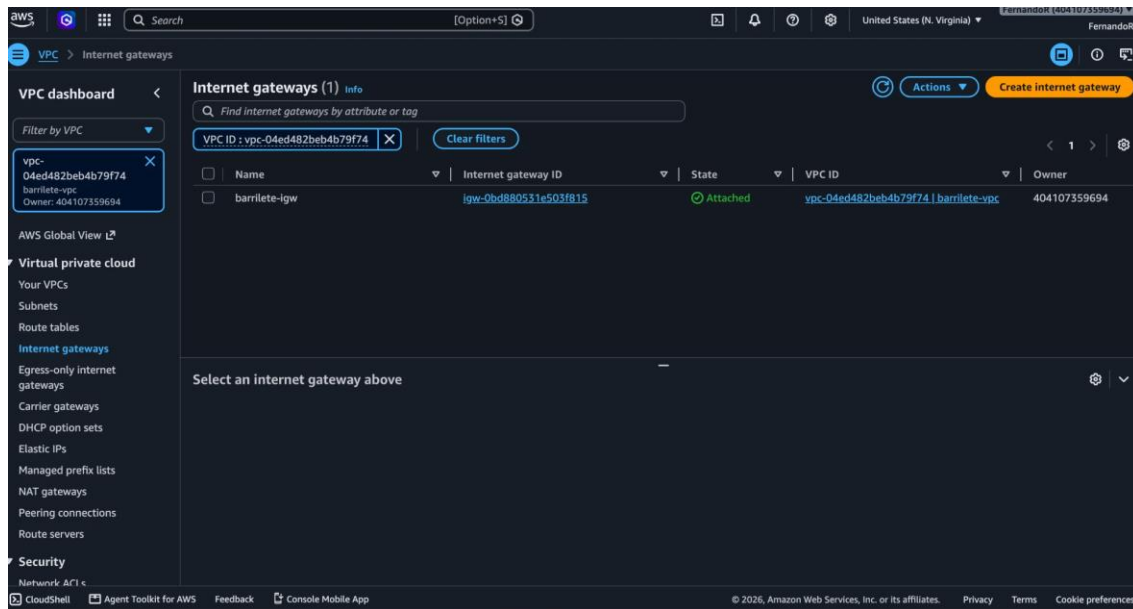
Aquí están las 5 subredes dentro de 192.168.10.0/24, todas en Available. Cada una corresponde a un área de la red.



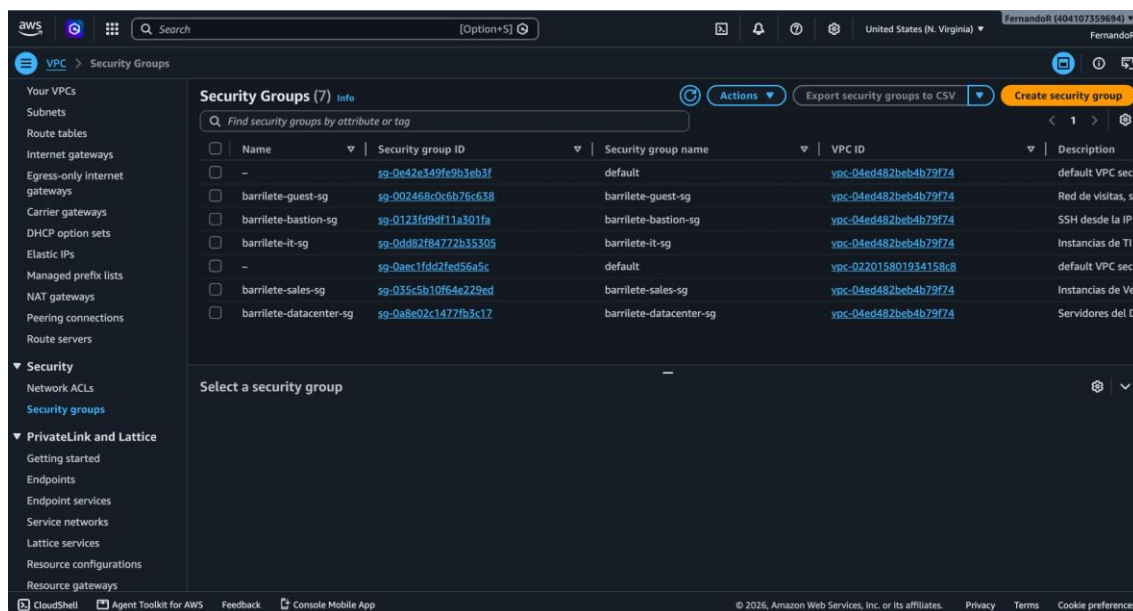
La tabla barrilete-private-rt está asociada a las 4 subredes privadas y solo tiene la ruta local 192.168.10.0/24, o sea que esas subredes no tienen salida a internet. La barrilete-public-rt solo va con la subred de management y la main se queda sin usar.



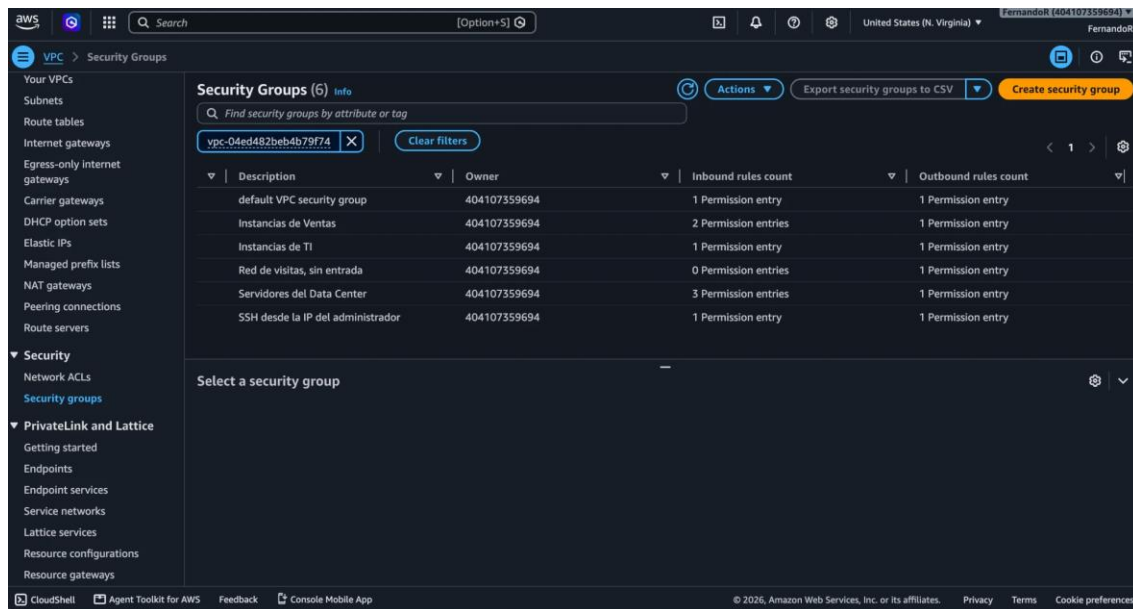
El Internet Gateway barrilete-igw está Attached a la VPC. Es la única salida a internet y solo la usa la subred pública.



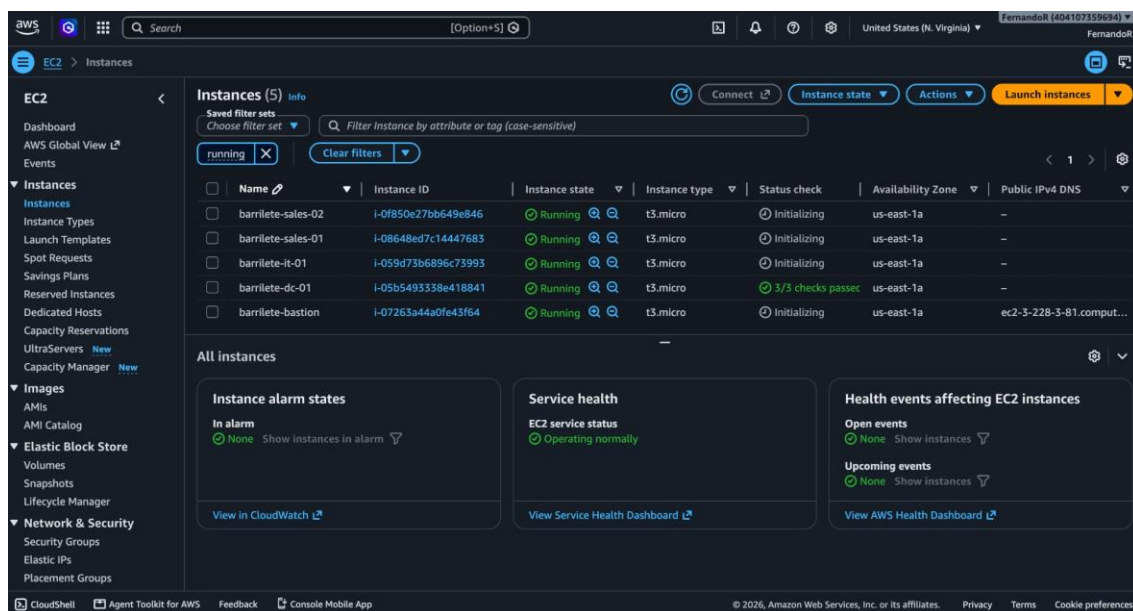
Estos son los security groups de la VPC: uno por área (guest, it, sales y datacenter) más el del bastion.



Acá se ve cuántas reglas tiene cada uno: el del data center tiene 3 de entrada, ventas 2, TI 1, el bastion 1 (SSH solo desde la IP del administrador) y visitas ninguna, así que nadie le puede entrar.



Por último, las 5 instancias t3.micro corriendo: sales-01, sales-02, it-01, dc-01 y el bastion, que es el único con IP pública.



Ejemplos de pruebas de conectividad

Prueba 1 (misma subred): entramos a sales-01 (192.168.10.10) pasando por el bastion y le hacemos ping a sales-02 (192.168.10.11), que está en la misma subred de ventas. Responde sin pérdida, así que dentro de ventas sí hay comunicación.


```
[fernandor@MacBook-Pro-de-Fernando-2 terraform % ssh -A -J ec2-user@3.235.87.197 ]
ec2-user@192.168.10.138

#_
~\   #####_      Amazon Linux 2023
~~\  _#####\
~~   \####|
~~~~  \|/
~~~~   V~!  ---> https://aws.amazon.com/linux/amazon-linux-2023

Last login: Fri Oct  2 02:44:17 2026 from 192.168.10.180
[ec2-user@ip-192-168-10-138 ~]$ ssh ec2-user@192.168.10.164
The authenticity of host '192.168.10.164 (192.168.10.164)' can't be established.
ED25519 key fingerprint is SHA256:v4RPFv3iLwuSC5CkTUTB4XLfgXpNjsB8ScbETW9PVS8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.10.164' (ED25519) to the list of known hosts

#_
~\   #####_      Amazon Linux 2023
~~\  _#####\
~~   \####|
~~~~  \|/
~~~~   V~!  ---> https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-192-168-10-164 ~]$ exit
logout
Connection to 192.168.10.164 closed.
[ec2-user@ip-192-168-10-138 ~]$
```